



by Kunal Sir

Constructor in Java:

A constructor in java is similar to a method that is invoked when an object of the class is created.

Need Of Constructor:

When you make various objects of a class, data members are automatically allocated under each object.

If you are allowed to initialize data members at the time of declaration in class then the data members corresponding to all the objects will possess the same initial values.

But in practice, you would want to have different initial values for the instance variable of the objects and in case you use any member method to initialize the data members you may have to call it separately every time after creating an object.

The creators of java have thus given us a boon by creating a concept called constructors. Hence, practically, you require such a member method that can automatically be called while creating an object to initialize its elements. To do so you need a CONSTRUCTOR. So let us dive deep into the constructors used in Java.

So, constructors are used to allocating values to the class variables during object creation, which is either explicitly done by the developer/programmer or by default constructor in Java itself.



Important Properties of Constructor:

Property-1: - Constructor name and the class name must be same-

```
public class A
{
    public A()
    {
        // statement for execution
    }
}
```

Property-2: - When our java class does not have any constructor then compiler put the default constructor in class-

```
public class A
{
}
}
```

```
public class A
{
    public A()
    {
    }
}
```

Property-3: - When our java class have any type of constructor (parameterized or non-parameterized) then compiler does not put the any type of constructor in class-

```
public class A
{
    public A()
    {
    }
}
```

```
public class A
{
    public A()
    {
    }
}
```



CJC

Complete Java Classes

by Kunal Sir

```
    }  
    }  
}
```

Property-4: - Constructor is used to initialize the instance variable of the class –

```
public class A  
{  
    int x = 10;  
}
```

```
public class A  
{  
    int x;  
    public A()  
    {  
        x = 10;  
    }  
}
```

Property-5: - We can write the multiple constructors in same class with different parameter-

```
public class A  
{  
    public A()  
    {  
        //statement for execution  
    }  
  
    public A(int x)  
    {  
        //statement for execution  
    }  
}
```



CJC

Complete Java Classes

by Kunal Sir

```
public class A(String s)
{
    //statement for execution
}

public class A(float f)
{
    //statement for execution
}
}
```

Property-6: - Constructor will get call automatically whenever the new object of class to be created-

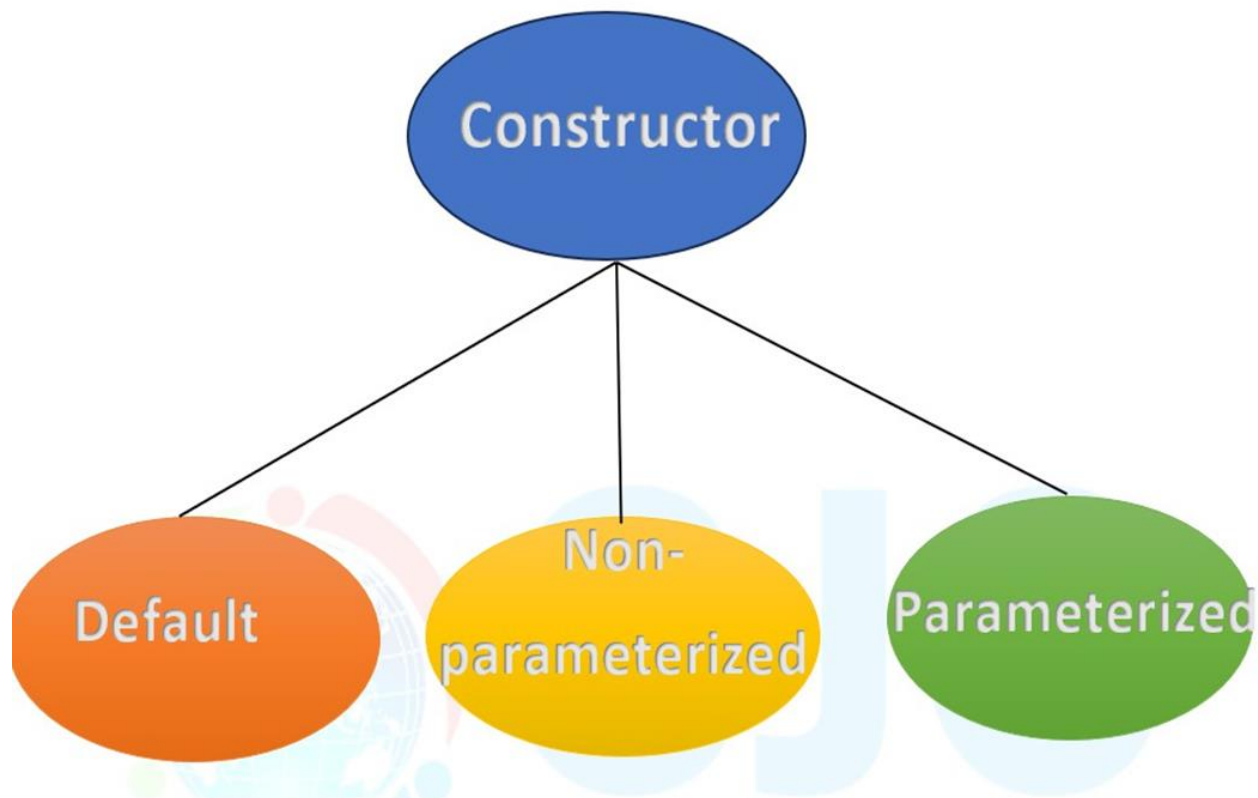
```
public class A
{
    public A()
    {
        System.out.println("Non-parameterized A");
    }

    public static void main(String args[ ])
    {
        A a = new A();
    }
}
```

Types of Constructors: -



1. Default Constructor
2. Non-Parameterized Constructor
3. Parameterized Constructor



1. Default Constructor: -

A default constructor is a constructor that has no parameters.

If there is no constructor in a class, compiler automatically creates a default constructor.



2. Non-Parameterized Constructor: -

Constructor without any argument is called a no-args constructor.

It's like overriding the default constructor and used to do some pre-initialization stuff such as checking resources, network connections, logging, etc.

3. Parameterized Constructor: -

A constructor which has one or more parameters to initialize the instance variables of a class.

One very easy example to understand parameterized constructors is given below-

Example 2:

```
public class Student
{
    int Number1, Number2;

    Student (int num1, int num2)
    {
        this.Number1 = num1;
        this.Number2 = num2;
    }

    public static void main(String[] args)
    {
        Student S = new Student(20, 50);
        System.out.println("Default value of Number1 is " + S.Number1);
        System.out.println("Default value of Number2 is " + S.Number2);
    }
}
```

Annotations:

- Class name:** Student
- Parameters:** int num1, int num2
- Initializes instance variables to custom value:** this.Number1=num1; this.Number2=num2;
- Creates object of Student class:** new Student(20,50);
- Calls parameterized constructor and passes argument to initialize instance variables:** Student S=new Student(20,50);

OUTPUT:

Default value of Number1 is 20
Default value of Number2 is 50